Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Automatică și Calculatoare
Departamentul Calculatoare și Tehnologia Informației

Computer Architecture
Laborator 5: Verilog Secvențial

Ștefan-Dan Ciocîrlan · 2026-2027

Cuprins

1. Automate de stări 1
   1.1. Maşina Mealy 1
   1.2. Maşina Moore 3
2. Memorie 4
   2.1. Registrul 4
   2.2. Memorie cu acces aleatoriu (RAM) 4

1. Automate de stări

Automatele de stări sunt absolut necesare în implementarea oricărui circuit digital. Există două tipuri de automate de stări, clasificate după tipurile de ieșiri generate de fiecare. Primul tip este **Maşina Mealy,** caracterizată de faptul că una sau mai multe ieşiri depind atât de starea curentă, cât şi de una sau mai multe intrări, iar al doilea este **Maşina Moore**, ale cărei ieşiri sunt doar o funcție care depinde doar de starea curentă.

1.1. Maşina Mealy

Un model general al unei maşini secvențiale Mealy este format dintr-o rețea combinațională care generează ieşirile, starea următoare şi o stare “Current State Register” reprezentând starea curentă, ca în figura de mai jos. Starea “Current State Register” este modelată utilizând bistabili D şi este sensibilă la semnalul de ceas (Clock). Atât ieşirea, cât şi determinarea stării următoare depind de intrare şi de starea curentă.

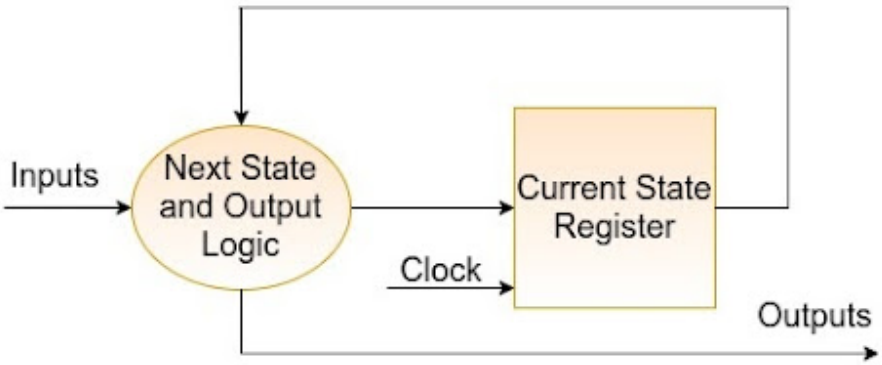


Figura 1: Mealy

1.1.1. Exemplu de automat de stări Mealy:

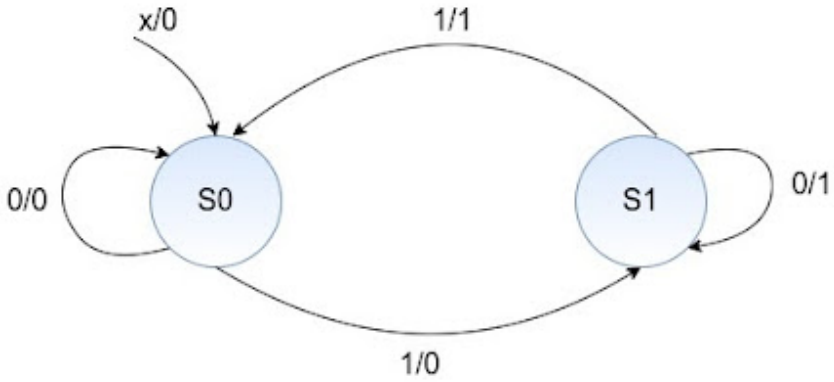


Figura 2: Exemplu Mealy

```
module mealy_fsm(
    output reg parity,
    input clk,
    input reset,
    input x);

    reg state, next_state;
    parameter S0=0;
    parameter S1=1;

    // Partea secvențială
    always @(posedge clk or negedge reset)
        if (!reset)
            state <= S0;
        else
            state <= next_state;

    // Partea combinațională
    always @(*) begin
        parity = 1'b0;
        case(state)
            S0:
                if(x)
                    next_state = S1;
                else
                    next_state = S0;
            S1:
                if(x) begin
                    parity = 1;
                    next_state = S0;
                end
                else begin
                    parity = 1;
                    next_state = S1;
                end
            default:
                next_state = S0;
        endcase
    end
endmodule
```

1.2. Maşina Moore

Un model general al unei maşini secvenţiale Moore este prezentat mai jos. Ieşirea sa este dependentă doar de blocul stării curente, iar starea următoare este determinată pe baza intrării şi a stării curente. În schema de mai jos, starea “Current State Register” este modelată utilizând bistabili D. Maşinile Moore obişnuite sunt descrise prin intermediul a trei blocuri, dintre care unul conţine logică secvenţială, iar celelalte două conţin logică de tip combinaţională.

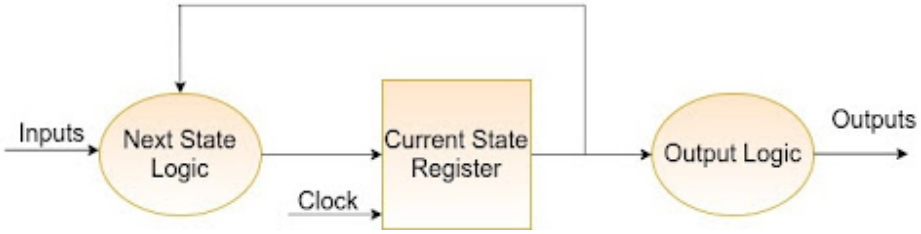


Figura 3: Moore

1.2.1. Exemplu de automat de stări Moore:

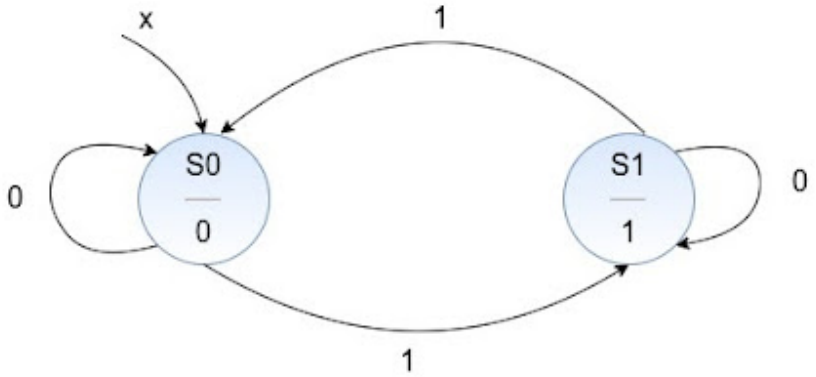


Figura 4: Exemplu Moore

```
module moore_fsm(
    output reg parity,
    input clk,
    input reset,
    input x);

    reg state, next_state;
    parameter S0=0;
    parameter S1=1;

     // Partea secvențială
    always @(posedge clk or negedge reset)
        if (!reset)
            state <= S0;
        else
            state <= next_state;

    always @(*) begin
        case(state)
            S0: begin
                parity = 0;
                if (x)
                    next_state = S1;
                else
                    next_state = S0;
```

```
            end
            S1: begin
                parity = 1;
                if(!x)
                    next_state = S1;
                else
                    next_state = S0;
            end
        endcase
    end
endmodule
```

2. Memorie

2.1. Registrul

Bistabilul este o celulă de memorie având capacitate de un bit, care poate fi utilizată pentru a stoca date digitale. Pentru a extinde capacitatea de stocare în ceea ce privește numărul de biți, se va utiliza un grup de bistabili, cunoscut și sub termenul de **registru**. Registrul de **n** biți este alcătuit din **n** bistabili și are capacitate de stocare de **n** biți.

Un registru, ca orice alt circuit secvențial este sensibil la schimbarea de front a semnalelor *clk* și *reset*. De asemenea, pentru a putea fi conectat la o magistrală, acesta are nevoie să execute două operații simple:

- citire - informația deja existentă în registru este preluată și eliberată pe ieșirea registrului
- scriere - informația aflată pe magistrală la un moment de timp se scrie în registru

Pentru o vizualizare mai clară a specificațiilor unui astfel de circuit digital secvențial, studiați interfața acestuia:

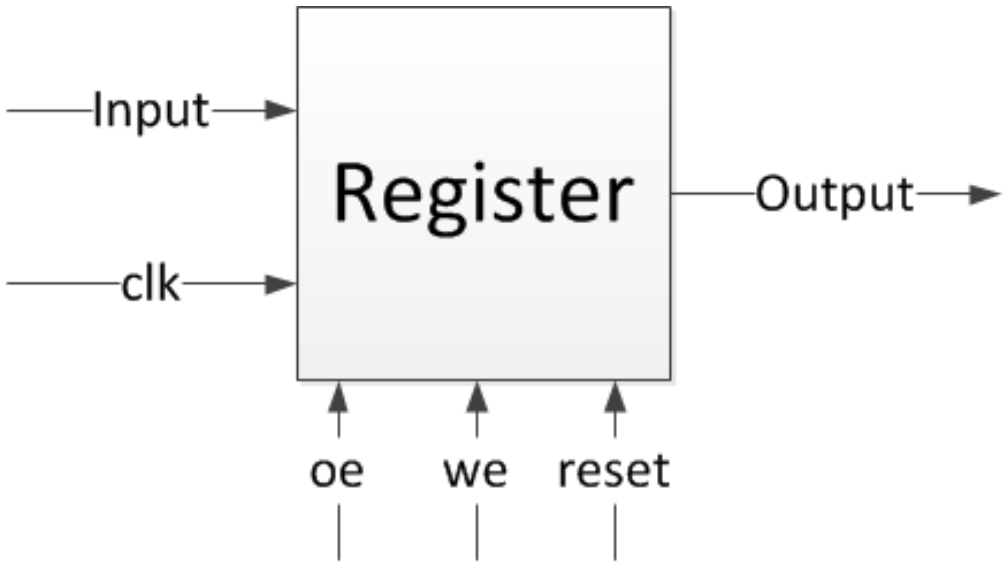


Figura 5: Registrul

2.2. Memorie cu acces aleatoriu (RAM)

Memoria poate fi asociată cu creierul uman, fiind folosită pentru a stoca date și instrucțiuni. Memoria unui calculator este spațiul de stocare din calculator unde sunt păstrate datele care urmează să fie procesate și instrucțiunile necesare pentru procesare. Aceasta se împarte în mai multe elemente cu caracteristici similare, numite celule. Fiecare celulă are o adresă unică numerotată de la 0 la N-1, unde N este dimensiunea blocului de memorie (numărul de celule din memorie).

Componenta hardware de tip memorie a unui calculator unde sunt stocate sistemul de operare, programele de bază și datele utilizate la momentul curent, pentru a fi accesate cu ușurință de procesor se numește **RAM** (Random Access Memory). RAM-ul este o memorie volatilă, însemnând că toate informațiile stocate în acesta vor fi pierdute la deconectarea calculatorului de la sursa electrică, urmând să fie recuperate la repornirea sistemului de pe HDD/SSD. RAM-ul este mic, atât ca dimensiune fizică, cât și din punct de vedere al capacității de stocare de date.

În comparație cu registrele, memoria de tip RAM este mai greu de accesat de către procesor. Fiind un circuit secvențial complex sunt necesari mai mulți cicli de ceas pentru a citi/scrie informația necesară. Totodată, oferă o capacitate mult mai mare de stocare, de care registrele nu dispun. Prin urmare, pentru implementarea eficientă a unui circuit digital, este foarte importantă gestionarea resurselor între memorie și registre, astfel încât să se permită stocarea și accesul la toate informațiile necesare într-un timp cât mai scurt.